Edge-AI Airspace Surveillance System

SS2026_Team_B1

Ikramul Hassan Sazid Muhammad Aamir Ejaz Ejaz Md Ruhul Kuddus Saleh

Electronic Engineering (B.Eng.)
Hamm-Lippstadt University of Applied Sciences, Lippstadt, Germany

1. Introduction

The Internet of Things (IoT) refers to a network of physical devices sensors, microcontrollers, and computers that collect data and communicate with each other over a network without human intervention. A Wireless Sensor Network (WSN) is a specific type of IoT system where distributed nodes equipped with sensors gather information about their environment and transmit it wirelessly to a central system for processing and monitoring.

In the domain of this project, these concepts are applied to **airspace surveillance and early-warning**. Instead of monitoring temperature or humidity like a typical WSN, our nodes monitor the sky detecting and classifying airborne objects, measuring their distance and speed, and raising an alarm when a threat is identified. Each node performs a specialized role and communicates wirelessly with the others through a shared messaging system, forming a small, distributed sensor network.

The target application is an **Edge-AI Airspace Surveillance System**: a distributed early-warning prototype that combines edge-based AI vision, distance and speed tracking, and remote alerting. A camera-equipped Raspberry Pi uses a YOLOv8 object-detection model to classify objects in the airspace as either safe (e.g. commercial aircraft, civilian helicopters) or hazardous (e.g. missiles, foreign drones). An Arduino node measures the distance and speed of detected objects, and the combined information is displayed on a live monitoring dashboard. When a hazardous object is detected, a remotely placed ESP32 node triggers a local alarm. All three nodes communicate through the lightweight MQTT publish/subscribe protocol.

2. Concept description

The system is a distributed early-warning prototype built from three hardware nodes that communicate wirelessly through a shared MQTT messaging layer. The Block Definition Diagram in Figure 1 shows how the overall system decomposes into its main blocks and their parts.

Node 1 (Raspberry Pi) is the central unit. it captures video through a USB camera, runs a YOLOv8 object-detection model to classify airborne objects as safe or hazardous, and hosts the MQTT broker and monitoring dashboard. Node 2 (Arduino UNO WiFi Rev2) is the sensor node, using an ultrasonic sensor to measure the distance and speed of a detected object. Node 3 (ESP32) is the remote alarm node, driving a KY-012 active buzzer when a threat is reported.

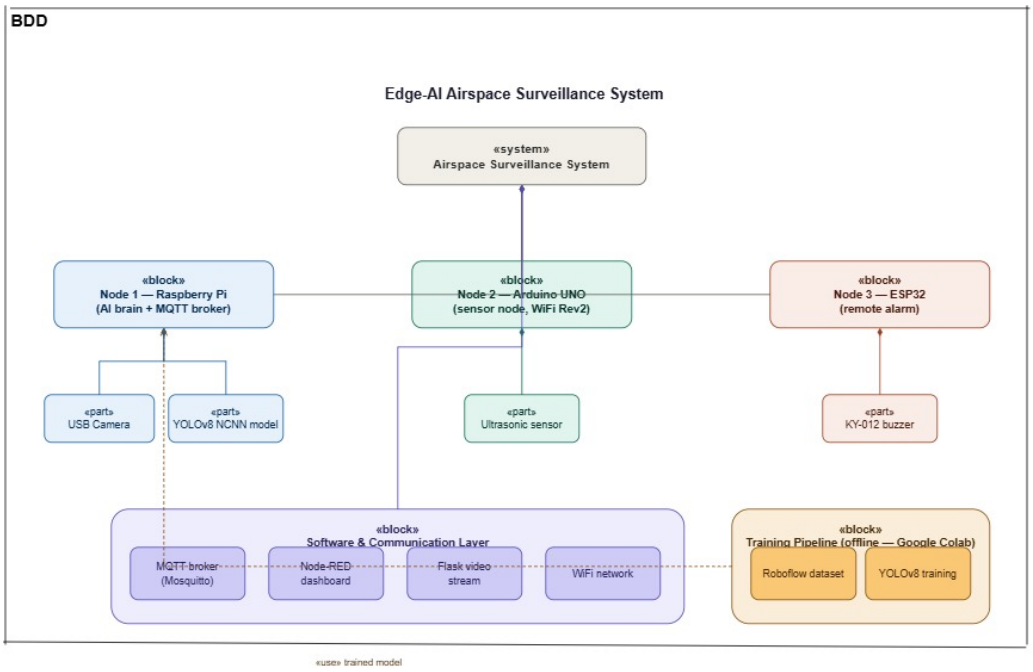

Figure 1: Block Definition Diagram of the Edge-AI Airspace Surveillance System.

The software and communication layer comprising the MQTT broker (Mosquitto), the Node-RED dashboard, the Flask video stream, and the WiFi network ties the nodes together and is treated as an integral part of the system. The training pipeline is shown as a separate, offline dependency rather than a component: it runs once in Google Colab to produce the trained YOLOv8 model that Node 1 later uses, and is not active while the system is running.

The main application is airspace surveillance: continuously monitoring the sky, identifying hazardous objects, displaying their type, distance, and speed on a live dashboard, and triggering a remote alarm when a threat is detected.

3. Project management

Project methods

The project was carried out using an incremental, task-based approach. Rather than building the whole system at once, the work was divided into independent nodes that could be developed and tested in parallel, then integrated through a common MQTT messaging layer. Each node was first tested on its own (for example, verifying that the ESP32 could receive a message and sound the buzzer using a public test broker) before connecting all nodes to the shared Raspberry Pi broker. Regular team coordination was used to agree on the common interface the MQTT broker address, port, and topic names so that the separately developed nodes would work together.

Task breakdown and management

The system was broken down along its three physical nodes, which mapped naturally onto the three team members. The shared elements the MQTT broker configuration, topic naming (for example `airspace/alert`), and the message format were agreed on jointly so that each member could work independently while staying compatible. Integration and testing of the complete system were done together once the individual nodes were working.

Roles and responsibilities

Team member	Node / role	Tasks
Ikramul Hassan Sazid	Node 1 Raspberry Pi	Set up the Raspberry Pi and Mosquitto broker; ran the YOLOv8 object-detection model on the camera feed; classified objects as safe or hazardous; published alerts and hosted the Node-RED dashboard.
Md Ruhul Kuddus Saleh	Nodes 1 & 2 Raspberry Pi & Arduino UNO REV2	Programmed Arduino telemetry; trained the YOLOv8 NCNN model; and assisted with Node 1 (Raspberry Pi) and Node-RED MQTT integration.
Muhammad Aamir Ejaz Ejaz	Node 3 ESP32	Programmed the ESP32 as an MQTT subscriber; connected it to the broker; received alert messages; and triggered the KY-012 active buzzer on a hazardous alert.

4. Technologies

This section describes the technological approaches used to implement the system, covering the sensors, communication protocols, programming languages, and supporting software.

Sensors and input devices

The system uses two main input devices. A USB camera connected to the Raspberry Pi provides the live video stream for object detection. An ultrasonic sensor connected to the Arduino measures the distance to a detected object, from which its speed is calculated. The ultrasonic sensor works by emitting a sound pulse and measuring the time for the echo to return, giving the distance to the object. Interfaces used include USB for the camera and the digital trigger/echo pins for the ultrasonic sensor.

Actuator

The output device is a KY-012 active buzzer module connected to the ESP32 through a digital GPIO pin (GPIO 5). As an active buzzer, it produces sound directly from a simple HIGH/LOW signal, without needing a separate tone-generation function.

Communication protocol

The nodes communicate using MQTT (Message Queuing Telemetry Transport), a lightweight publish/subscribe messaging protocol designed for IoT applications. A central broker (Eclipse Mosquitto, running on the Raspberry Pi) receives all published messages and forwards them to the subscribed clients. This decouples the devices a publisher does not need to know the address of any subscriber; it only publishes to a named topic, and any client subscribed to that topic receives the message. The project uses three topics: one for the detection state sent to the dashboard, one for the sensor telemetry from the Arduino, and one for the alert sent to the ESP32. MQTT runs over the WiFi network on the default port 1883.

Programming languages

The Raspberry Pi node is programmed in Python, using the Ultralytics library for the YOLOv8 model and the Paho MQTT library for messaging. The Arduino and ESP32 nodes are programmed in C++ using the Arduino framework. The ESP32 uses the WiFi library for network connectivity and the PubSubClient library for MQTT communication.

Supporting software

Additional software includes Node-RED, used to build the live monitoring dashboard that displays object type, distance, speed, and threat status; and a Flask web server on the Raspberry Pi that streams the annotated video feed. The YOLOv8 object-detection model was trained separately using Google Colab and the Roboflow dataset platform, then exported to the lightweight NCNN format for efficient execution on the Raspberry Pi.

5. Implementation

Static structure of the environment

The system is organised as three independent modules (nodes) that share a common MQTT communication layer, rather than as classes within a single program. Each node runs its own firmware or software and connects to the central Mosquitto broker hosted on the Raspberry Pi. The static structure is therefore best described as a distributed set of cooperating modules.

The Raspberry Pi module runs the vision and control software (object detection, classification, dashboard, broker). The Arduino module runs the sensor firmware (distance and speed measurement). The ESP32 module runs the alarm firmware (message subscription and buzzer control). The three modules are connected only through named MQTT topics, which keeps them loosely coupled — each can be developed, replaced, or restarted without affecting the others.

State diagram ESP32 alarm node (Node 3)

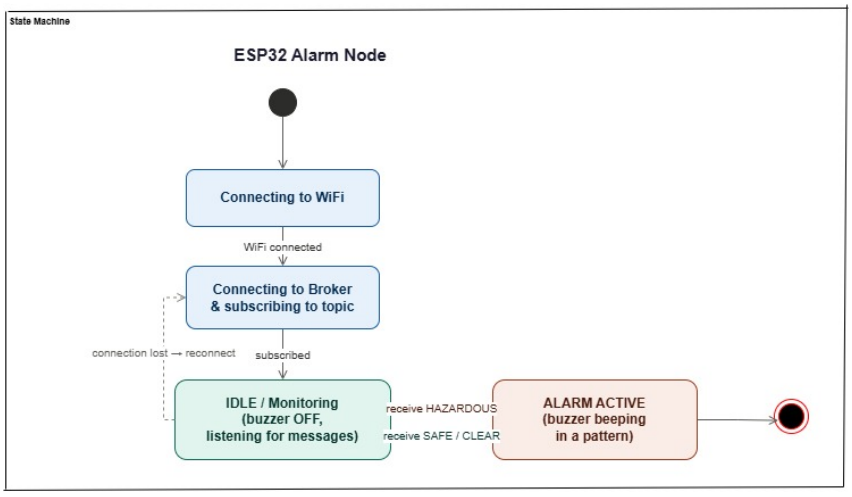

Figure 2: State diagram of the ESP32 alarm node.

Because the ESP32 node is event-driven, its behaviour is best represented with a state diagram (Figure 2). After power-up, the node moves through an initialisation sequence connecting to

WiFi, then connecting to the broker and subscribing to the alert topic. It then enters the IDLE / Monitoring state, where the buzzer is off and the node listens for incoming messages. When a HAZARDOUS message is received, it transitions to the ALARM ACTIVE state and the buzzer beeps in a repeating pattern. When a SAFE or CLEAR message is received, it returns to the IDLE state and the buzzer is silenced. If the network or broker connection is lost, the node automatically returns to the connection stage and reconnects.

Use cases

The main use case is automated threat detection and alerting. An operator sets up the system to monitor an area of airspace. The camera continuously observes the sky, and the Raspberry Pi classifies detected objects as safe or hazardous while the Arduino reports their distance and speed. All information is displayed live on the Node-RED dashboard. When a hazardous object is detected, the ESP32 placed in a separate location such as a control room sounds an alarm, allowing personnel to respond even if they are not watching the dashboard. A secondary use case is live monitoring, where the operator uses the dashboard and video stream simply to observe and track all objects in the monitored airspace in real time.

6. Results

The system was successfully implemented and tested as a complete distributed prototype. All three nodes connected to the shared MQTT broker on the Raspberry Pi and communicated correctly through their assigned topics. The results below describe the behaviour of each part and the integrated system.

Object detection and classification (Node 1)

The YOLOv8 model running on the Raspberry Pi detected and classified objects in the camera feed in real time, labelling each as safe or hazardous. The annotated video stream, showing bounding boxes, object type, and the current system status, was served over the Flask web server and viewed in a browser.

Figure 3: Live detection feed. Hazardous objects set the state to THREAT_DETECTED (red); safe objects show CLEAR (green).

Distance and speed measurement (Node 2)

The Arduino node measured the distance to a target with the ultrasonic sensor and calculated its speed, publishing the values via MQTT. Median filtering was applied to reduce sensor noise, giving stable readings.

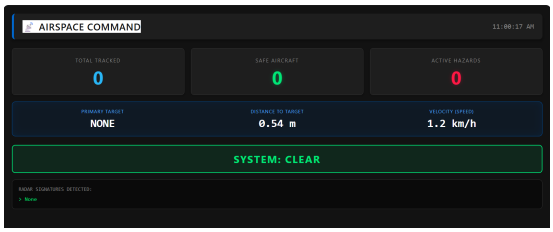

Figure 4: Arduino telemetry showing measured distance and speed.

Live dashboard

The Node-RED dashboard displayed all information together object type, distance, speed, number of tracked objects, and the overall threat status. The status banner changed colour and pulsed red when a hazard was detected.

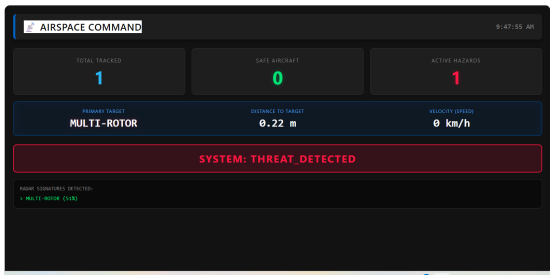

Figure 5: Node-RED monitoring dashboard.

Alarm response (Node 3 ESP32)

The ESP32 correctly subscribed to the alert topic and responded to messages from the Raspberry Pi. When a HAZARDOUS message was published, the ESP32 activated the KY-012 buzzer, which sounded until a SAFE / CLEAR message was received. The Serial Monitor confirmed the connection sequence and the messages received.

```
WiFi Connected! IP: 10.40.147.62
Connecting to MQTT Broker...Connected!
Subscribed to topic: airspace/alert
----------------------------
Topic   : airspace/alert
Message: CLEAR
✅ All Clear. Alarm Disarmed
----------------------------
Topic   : airspace/alert
Message: HAZARDOUS
⚠️ THREAT DETECTED! Alarm Armed
----------------------------
Topic   : airspace/alert
Message: CLEAR
✅ All Clear. Alarm Disarmed
```

Figure 6: ESP32 Serial Monitor confirming connection and a received alert.

Integrated system test

In the final test, the full chain was verified end to end: a hazardous object shown to the camera was detected by the Pi, its distance and speed were reported by the Arduino, the dashboard updated to show the threat, and within a fraction of a second the ESP32 buzzer sounded. When the object was removed, the status returned to CLEAR and the buzzer stopped. This confirmed that the publish/subscribe architecture worked reliably and that all three nodes were correctly integrated.

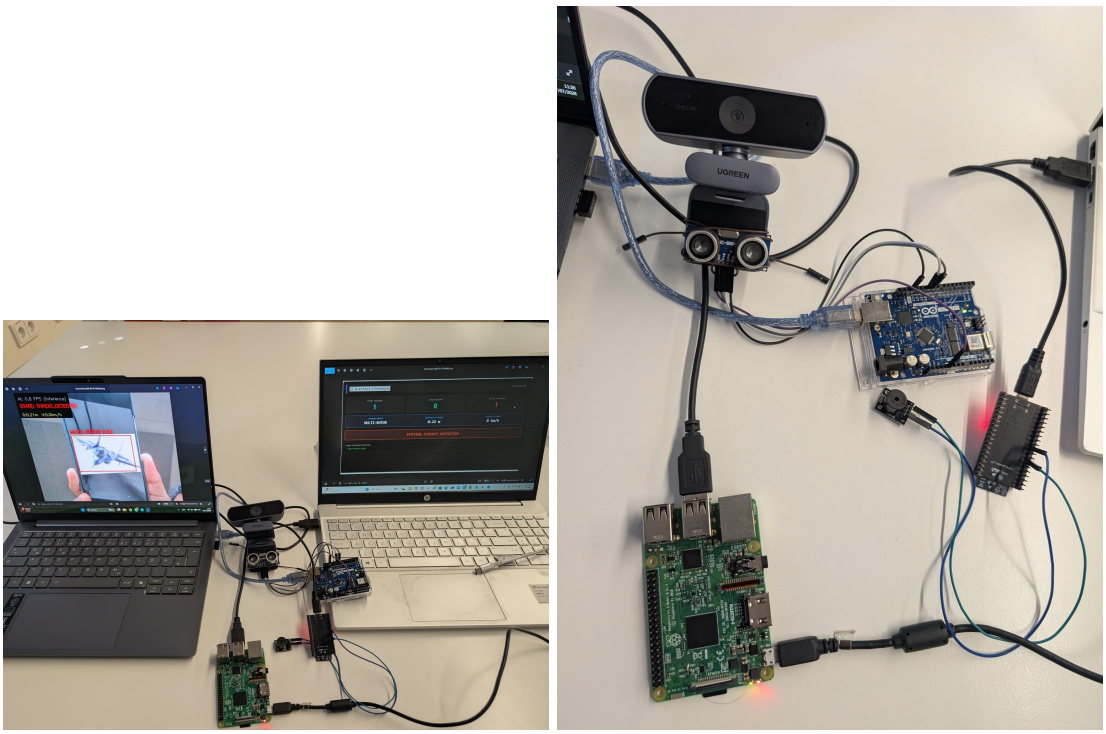

Figure 7: Integrated Overall System.

The results showed that MQTT is well suited to this kind of distributed early-warning system: the loose coupling allowed each node to run independently while still cooperating, and the alert reached the remote ESP32 quickly and reliably. A limitation observed was that using a public/local broker on an open topic could receive unintended messages, so a unique topic name and,

for a real deployment, TLS security and authentication would be recommended. The ultrasonic sensor also had a limited range, suitable for bench testing rather than real airspace distances.

7. Sources / References

- IoT definition: https://www.geeksforgeeks.org/computer-networks/introduction-to-internet-of-th
- Eclipse Mosquitto — MQTT broker documentation: https://mosquitto.org/documentation/
- Ultralytics YOLOv8: https://docs.ultralytics.com/
- PubSubClient (Arduino MQTT library): https://github.com/knolleary/pubsubclient
- Node-RED: https://nodered.org/
- Roboflow (dataset platform): https://roboflow.com/
- Project repository (GitHub): `[ insert your GitHub link here ]`